

ESC113: Course Roadmap

Numerical Methods using MATLAB for Linear Systems, Nonlinear Equations, ODEs, PDEs and Integration

1 Module 1: Linear Systems $Ax = b$

1.1 Course Overview

Topics covered: (i) least squares, (ii) cubic splines (local interpolation), (iii) Gauss elimination, (iv) LU decomposition.

1.2 Gauss Elimination

Forward elimination (cost $\sim O(n^3)$):

```
for k = 1:n-1
    for i = k+1:n
        m = a(i,k)/a(k,k);
        b(i) = b(i) - m*b(k);
        for j = k+1:n
            a(i,j) = a(i,j) - m*a(k,j);
        end
    end
end
```

Purpose:

Transform the system to an upper triangular form $Ux = y$.

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \rightarrow U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix}$$

Backward substitution (cost $\sim O(n^2)$):

$$x_n = \frac{b_n}{a_{nn}}, \quad x_k = \frac{1}{a_{kk}} \left[b_k - \sum_{j=k+1}^n a_{kj} x_j \right], \quad k = n-1, \dots, 1.$$

Total cost: $O(n^3) + O(n^2) \approx O(n^3)$.

1.3 LU Decomposition

Store the multipliers $m_{ik} = a_{ik}/a_{kk}$ in a unit lower-triangular matrix L ; the reduced matrix is U .

$$Ax = b \rightarrow LUx = b, \quad Ux = y, \quad Ly = b.$$

- **Forward substitution:** $Ly = b$, $y_k = b_k - \sum_{j=1}^{k-1} l_{kj} y_j$ (with unit diagonal).
- **Backward substitution:** $Ux = y$.
- **Advantage:** factorize once, then solve for many right-hand sides cheaply.

1.4 Partial Pivoting

Worked example: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix} x = \begin{bmatrix} 3 \\ 4 \\ 4 \end{bmatrix}$. After $R_2 \rightarrow R_2 - R_1$, $R_3 \rightarrow R_3 - 2R_1$ the pivot $a_{22} = 0$,

so rows 2 and 3 must be swapped.

Rule: for each column k , find $j \geq k$ with largest $|a_{jk}|$ and interchange rows j and k (track swaprow and maxval).

Purpose:

Avoids zero (or very small) pivots and improves numerical stability.

1.5 Iterative Methods

Jacobi (cost $\sim O(n^2)$ per iteration): $x_k^{(i+1)} = \frac{1}{a_{kk}} \left[b_k - \sum_{j \neq k} a_{kj} x_j^{(i)} \right]$.

Gauss Seidel: uses the newest values immediately,

$$x_k^{new} = \frac{1}{a_{kk}} \left[b_k - \sum_{j=1}^{k-1} a_{kj} x_j^{new} - \sum_{j=k+1}^n a_{kj} x_j^{old} \right].$$

Purpose:

Iteratively improve the solution until convergence.

Stopping criteria:

- $\epsilon_1 = \|x^{i+1} - x^i\|_2 \leq \text{TolX}$
- $\epsilon_2 = \|b - Ax^{i+1}\|_2 \leq \text{TolFun}$

Key Takeaways

Direct methods: exact up to round-off, $O(n^3)$. Iterative methods: cheaper per sweep, need a stopping tolerance and convergence. Pivoting fixes zero (or tiny) pivots.

2 Module 2: Nonlinear Equations

2.1 Newton's Method & Fixed-Point View

Newton's method:
$$x^{k+1} = x^k - \frac{f(x^k)}{f'(x^k)} = g(x^k), \quad x^* = g(x^*).$$

Subtracting and using the Mean Value Theorem:

$$x^{k+1} - g(\xi_k)x^k, \quad e^k = x^k - x^*.$$

Convergence guarantees: $|g'(\xi)| < 1$. For Newton, $g'(x) = \frac{f(x)f''(x)}{(f'(x))^2}$ so we need $f'(x) \neq 0$ and f approaching 0 faster than f' near the root.

Quadratic convergence (Taylor expansion about x^*):
$$e^{k+1} = \frac{f''(\xi_k)}{2f'(x^*)}(e^k)^2.$$

- Key idea:**
- Newton's method can be viewed as a fixed-point iteration $x^{k+1} = g(x^k)$.
 - Converges quadratically near the root if conditions are satisfied.

2.2 Newton–Raphson for Systems

For $R(\mathbf{x}) = 0$ with vector of functions R and unknown $\mathbf{x}$, linearize with the Jacobian:

$$J(\mathbf{x}^k)(x^{k+1} - \mathbf{x}^k) = R(\mathbf{x}^k), \quad J_{ij} = \frac{\partial R_i}{\partial x_j}.$$

Each iteration is a linear solve: $A\mathbf{x} = \mathbf{b}$ with $A = J$, $\mathbf{x} = \Delta\mathbf{x}$, $\mathbf{b} = -R$ (this links Module 1 to Module 2).

- Applications:**
- Systems of nonlinear equations (e.g., from discretized PDEs).
 - Each Newton step requires solving a linear system.

2.3 Pseudo-code for NR

```
Example:  $R_1 = ax_1^2$  for  $1 < i < 2$ ;  $R_2 = \sin x_1 - x_1 \cos x_2$ .  
  
function J = myjacobian(x)    % analytic (or numerical) Jacobian  
function R = myfun(x)        % residual vector  
ab = <initial guess>;  
for k = 1:maxiter  
    R = myfun(ab); J = myjacobian(ab);  
    dx = J \ (-R);           % linear solve  
    ab = ab + dx;  
    if norm(R) < tol, break, end  
end
```

- Notes:**
- Jacobian can be analytic or approximated numerically.
 - Use a stopping tolerance (e.g., $\text{norm}(R) < \text{tol}$).
 - Each iteration solves a linear system.

3 Module 3: Numerical Differentiation

3.1 Finite-Difference Formulas

Used when the Jacobian or a PDE/ODE derivative must be approximated. Derived from Taylor series.

Scheme	Formula	Accuracy
Forward	$f'(x) \approx \frac{f(x + \Delta x) - f(x)}{\Delta x}$	$O(\Delta x)$
Backward	$f'(x) \approx \frac{f(x) - f(x - \Delta x)}{\Delta x}$	$O(\Delta x)$
Central	$f'(x) \approx \frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x}$	$O(\Delta x^2)$
Second derivative	$f''(x) \approx \frac{f(x + \Delta x) - 2f(x) + f(x - \Delta x)}{\Delta x^2}$	$O(\Delta x^2)$

- Key points:**
- Derived using Taylor series expansions.
 - Central difference is more accurate.
 - Used for numerical Jacobians and PDE/ODE discretization.

The second-derivative stencil comes from adding the forward and backward Taylor expansions, weighing $\alpha_1 = -1/\Delta x^2$, $\alpha_2 = 2/\Delta x^2$.

3.2 Motivating PDEs

Heat/diffusion (linear): $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$. Reaction diffusion (nonlinear): $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} - Ku^2$.

- Notes:**
- Finite differences approximate spatial derivatives.
 - Forms the basis for numerical solution of PDEs.

4 Module 4: ODE Initial Value Problems

4.1 Setup

n -th-order ODE: $y^{(n)} = f(x, y, y', \dots, y^{(n-1)})$, First-order IVP: $y' = f(x, y)$, $y(x_0) = y_0$.
Grid: $h = \frac{x_f - x_0}{N}$, $x_i = x_0 + ih$. Error sources: round-off and truncation.

Goal:

- Approximate the solution $y(x)$ at discrete points.
- Understand accuracy, stability and step-size effect.

4.2 Euler-Type Methods

Method	Update	Local err.	Global err.
Explicit Euler	$y_{i+1} = y_i + hf(x_i, y_i)$	$O(h^2)$	$O(h)$
Implicit Euler	$y_{i+1} = y_i + hf(x_{i+1}, y_{i+1})$	$O(h^2)$	$O(h)$
Predictor Corrector (Heun)	$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})]$	$O(h^3)$	$O(h^2)$

Explicit Euler is cheap; implicit Euler needs a (possibly nonlinear) solve each step, so use NR, but it is very stable.
Predictor: $\tilde{y}_{i+1} = y_i + hf(x_i, y_i)$. Corrector: $y_{i+1} = y_i + \frac{h}{2} [f(x_{i+1}, \tilde{y}_{i+1})]$, then average.

Takeaway:

- Explicit: simple, lower accuracy, conditionally stable.
- Implicit: more expensive, better stability (especially for stiff problems).
- Predictor–Corrector: higher accuracy with small extra cost.

4.3 Runge Kutta Family

General form: $y_{i+1} = y_i + h\phi$, $\phi = a_1k_1 + a_2k_2 + \dots$, with $k_1 = f(x_i, y_i)$, $k_2 = f(x_i + p_1h, y_i + q_1hk_1), \dots$

- RK1 – explicit Euler:** $y_{i+1} = y_i + hk_1$.
- RK2:** match with the Taylor series $\rightarrow a_1 + a_2 = 1$, $a_2p_1 = \frac{1}{2}$, $a_2q_1 = \frac{1}{2}$.
Choose $a_1 = a_2 = \frac{1}{2}$, $p_1 = q_1 = 1$ gives $y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_i + h, y_i + hf(x_i, y_i))]$; local error $O(h^3)$.
- RK4:** $y_{i+1} = y_i + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$.

Why RK?

- Higher accuracy for the same step size.
- RK4 is very popular and effective for many problems.
- Constructed by matching Taylor series coefficients.

4.4 Numerical Stability

Test problem: $y' = \lambda y$, $y(0) = y_0$, true solution $y = e^{\lambda x}y_0$.

- Explicit Euler:** $y_i = y_0(1 + \lambda h)^i$. Stable if $|1 - \lambda h| \leq 1 \iff h \leq \frac{2}{\lambda}$ (a stability limit).
- Implicit Euler:** $y_i = \frac{y_0}{(1 - \lambda h)^i}$. Stable for all $h > 0$ ($\rightarrow$ unconditionally stable), but stability does not imply accuracy.
- RK2/RK4** are analyzed the same way. For nonlinear ODEs, linearize locally (e.g. $e \approx g = \frac{\partial f}{\partial y} \big|_{y^*}$).

Key points:

- Stability restricts the step size (explicit methods).
- Implicit methods are unconditionally stable, but may still be inaccurate if h is too large.
- Always consider both stability and accuracy.

4.5 Systems of ODEs & Stiffness

- Vector form:** $y_{i+1} = y_i + hF(x_i, y_i)$; implicit versions need Newton–Raphson.
- Higher-order ODEs** convert to first-order systems: $z_1 = y$, $z_2 = y'$, $z_3 = y''$, $z'_3 = f(x, y)$.
- Stiff system example:** $y'_1 = -98y_1 + 1998y_2$, $y'_2 = -999y_1 - 1999y_2$ with eigenvalues -1 and -1000 . The fast mode e^{-1000x} dies quickly but forces tiny h for explicit methods.
- Remedies:** implicit methods, non-uniform step size (small h where variation is large).

Stiff problems:

- Multiple time scales (fast and slow dynamics).
- Explicit methods require very small h .
- Use implicit methods or adaptive/non-uniform step size.

5 Module 5: ODE Boundary Value Problems

5.1 Boundary Conditions

- **Dirichlet (1st type):** $y(x_a) = \text{value}$.
- **Neumann (2nd type):** $y'(x_a) = \text{value}$.
- **Robin/Mixed (3rd type):** $\alpha y' + \beta y = 0$.

Note:

- Boundary conditions specify the solution at the boundaries.
- Mixed (Robin) combines Dirichlet and Neumann types.

5.2 Finite-Difference Solution

Example: $\frac{d^2y}{dx^2} = \phi$, $\phi_a(x) = 0$, $\alpha_b(y) = 1$.

1. Uniform grid: $x_i = a + (i - 1)h$, $h = \frac{b - a}{N - 1}$, N unknowns.
2. Interior nodes $i = 2, \dots, N - 1$: $\phi_i = \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2}$ ($N - 2$ equations).
3. Boundary conditions add 2 equations $\rightarrow N$ equations, solved by Gauss elimination (or NR if nonlinear).

Neumann at x_b : use a fictitious node $\frac{y_{N+1} - y_{N-1}}{2\Delta x} = 0$, and also enforce the ODE at node N to close the system.

Key points:

- Discretization converts the BVP into a system of algebraic equations.
- Boundary conditions replace equations at the first and last nodes.
- Use Gauss elimination for linear problems or Newton–Raphson for nonlinear problems.

6 Module 6: Partial Differential Equations

6.1 1D Transient Diffusion — Method of Lines

$\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2}$ with $u(x, 0) = 0$, $u(0, t) = 0$, $u(L, t) = 0$ (or a Neumann condition).

1. Discretize in space: $\Delta x = h = \frac{L}{N + 1}$, giving $N - 2$ ODEs

$$\frac{du_i}{dt} = \alpha \frac{y_{i-1} - 2u_i + y_{i+1}}{\Delta x^2}, \quad i = 2, \dots, N - 1.$$

2. Integrate in time as a system of ODE: IVPs (Euler, RK, ...).
3. Stability: for explicit Euler / RK4, $h \leq h_{\text{stability}}$, roughly $\frac{2\Delta x^2}{\alpha \Delta t_{\text{tmax}}}$.

Matrix form: $\frac{dU}{dt} = \alpha AU$. Diagonalizing $A = X\Lambda X^{-1}$ decouples the system into $\frac{d\hat{c}}{dt} = \alpha \Lambda \hat{c}$, so modes evolve as $\hat{c}_k(t) = \hat{c}_k(0)e^{\alpha \lambda_k t}$.

Key ideas:

- Method of lines: discretize space $\rightarrow$ system of ODEs in time.
- Use standard ODE solvers (Euler, RK, ...).
- Stability of explicit methods imposes a time step restriction.
- Eigen decomposition helps analyze behavior and long-term solution.

6.2 2D Steady-State Laplace Equation

$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$ with Dirichlet data T_1, T_2, T_3, T_4 on the four sides.

- Grid $\Delta = h = \frac{L}{N + 1}$, ($\Delta x = \Delta y$ is a convenience, not a requirement).

Discretization (interior nodes):

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0.$$

6.3 Nonlinear PDE & Term Project

Reaction diffusion: $\frac{\partial(C_A)}{\partial t} = D \frac{\partial^2 C_A}{\partial x^2} + kC_A^2 = 0$ on the unit square with $C_A = 1$ on all edges (reaction $2A \rightarrow B$). Discretize as above and solve with Newton–Raphson.

Project deliverables:

- MATLAB code with comments.
- Report of < 3 pages: introduction, problem description, results, and conclusion.

Notes:

- Nonlinear PDEs require iterative solvers (e.g., Newton–Raphson).
- Discretization leads to a large system of nonlinear equations.
- Clearly present assumptions, results and discussion.

7 Module 7: Numerical Integration & Interpolation

7.1 Idea

$I = \int_a^b f(x) \, dx \approx \int_a^b \bar{f}(x) \, dx$, where $\bar{f}$ is an interpolant through data (x_i, f_i) .

Split the domain $[a, b]$ into subintervals and sum:

$$I = I_1 + I_2 + \cdots + I_n.$$

Key idea:

- Approximate the integrand by an interpolation polynomial and integrate it.
- Leads to standard quadrature rules (trapezoidal, Simpson, etc.).

7.2 Interpolation Ladder

1. **Linear:** $f(x) = ax + b$, solve $\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$.

2. **Quadratic:** the Vandermonde approach $(ax^2 + bx + c)$ does not generalize well and is harder to compute, so use Newton form:

$$f(x) = b_0 + b_1(x - x_1) + b_2(x - x_1)(x - x_2),$$

$$b_0 = f(x_1), \quad b_1 = f[x_1, x_2] = \frac{f(x_2) - f(x_1)}{x_2 - x_1}, \quad b_2 = f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}.$$

Uniform spacing h : $b_1 = \frac{f_2 - f_1}{h}, \quad b_2 = \frac{f_3 - 2f_2 + f_1}{2h^2}.$

3. **nth order:** $f_n(x) = b_0 + b_1(x - x_1) + \cdots + b_n \prod_{i=1}^n (x - x_i)$, with divided differences

$$b_n = f[x_1, \dots, x_{n+1}] = \frac{f[x_2, \dots, x_{n+1}] - f[x_1, \dots, x_n]}{x_{n+1} - x_1}. \quad \text{Uniform: } \frac{1}{n!} \frac{\Delta^n f}{h^n} \approx \frac{1}{n!} \frac{d^n f}{dx^n}.$$

Takeaways:

- Use Newton form for higher-order interpolation (more stable).
- Divided differences work for non-uniform grids.
- Integrating the interpolant over subintervals leads to standard numerical integration formulas.
- Choose interpolation order based on accuracy requirements and data smoothness.

Integrating these interpolants over subintervals gives the trapezoidal and Simpson-type rules.

Concept Map: What Reuses What

Tool	Used in
Gauss / LU	Every implicit scheme, Newton–Raphson steps, BVPs, Laplace equation
Newton–Raphson	Nonlinear systems, implicit Euler for nonlinear ODEs, nonlinear PDE (term project)
Finite differences	Jacobian by numerical differentiation, ODE–IVP, BVP, PDE
Stability analysis ($y' = \lambda y$)	Choosing h for Euler/RK, method of lines
Interpolation	Splines, integration, finite-difference stencils